INFORME TÉCNICO

Análisis y mejora de seguridad de una aplicación Android

Proyecto: SistemaSeguridadApp-MapGPS

Estudiantes: Goran Almendra – Sergio Álvarez – Juan Cirternas – Michael Hernández

Asignatura: Programación Android

Docente: Oscar Enrique Monárdez Saguas

Institución: Instituto Profesional Santo Tomás Ovalle

Fecha de entrega: 28/10/2004]

1. Introducción

La seguridad de las aplicaciones móviles es fundamental para proteger la información de las personas usuarias, mantener la integridad de los datos y reducir el riesgo de accesos no autorizados. Este informe presenta el análisis técnico y la documentación de las medidas de seguridad aplicadas en una aplicación Android basada en servicios geoespaciales (Google Maps), partiendo de una evaluación inicial en la plataforma MobSF que arrojó hallazgos críticos y calificaciones de riesgo elevadas, lo cual motivó la modificación del código hasta alcanzar una puntuación de 73, posicionándose en el rango A de seguridad.

El README del proyecto identifica tres áreas principales de riesgo: inyección SQL, comunicación no segura y exposición de datos sensibles. Asimismo, declara las mejoras implementadas para atenuar estas vulnerabilidades, tales como el cifrado robusto de la información, el uso exclusivo de HTTPS y la validación y sanitización de entradas. La efectividad de cada medida correctiva se verifica mediante la revisión de código, la protección de credenciales y claves de API, y las pruebas documentadas presentadas en este informe.

2. Objetivos

2.1 Objetivo general

Analizar y documentar las vulnerabilidades de seguridad de una aplicación Android y las medidas aplicadas para mitigarlas.

2.2 Objetivos específicos

- Identificar y describir las vulnerabilidades contempladas en el README.md.
- Explicar las medidas técnicas aplicadas para reducir los riesgos.
- Documentar las pruebas realizadas y sus resultados con evidencias verificables.
- Revisar la documentación complementaria y proponer acciones de mejora continua.

3. Descripción del proyecto

Nombre del proyecto: Sistema SeguridadApp (Paquete: `com.example.sistemaseguridadapp`).

Propósito y funcionalidad principal: Aplicación móvil orientada al control y gestión de seguridad, la cual maneja inicio de sesión, ubicación geográfica y captura de registros mediante cámara para la supervisión de accesos y zonas.

Tecnologías utilizadas: Android Studio, Java (versión de Android SDK Mínimo: 30, Objetivo: 37), SQLite, y herramientas de análisis estático MobSF v4.6.1.

Alcance del análisis: Se revisó la actividad de inicio de sesión (`LoginActivity`), el módulo interactivo de mapas con geolocalización (googleMaps), la captura de registros mediante la cámara y el almacenamiento local cifrado de credenciales de usuario.

4. Vulnerabilidades identificadas

4.1 Inyección SQL

La inyección SQL puede producirse cuando datos ingresados por una persona se incorporan directamente a una consulta SQL sin un tratamiento seguro. Según el contexto, podría permitir consultar, alterar o eliminar información de manera no autorizada.

Análisis del proyecto:

• Componente o archivo revisado: com/example/sistemaseguridadapp/LoginActivity.java y defpackage/od.java.

• Código o comportamiento vulnerable observado: Se detectó el uso de consultas SQLite sin procesar (raw queries) donde entradas no confiables podrían concatenarse directamente en la sentencia.

• Impacto potencial: Permitiría a un usuario malintencionado manipular las consultas SQLite locales para realizar un salto de autenticación (bypass), comprometiendo las credenciales de acceso y la información almacenada en el dispositivo.

Medida recomendada: utilizar consultas parametrizadas o sentencias preparadas, validar los datos y aplicar privilegios mínimos en la base de datos. Si la aplicación consume una API, revisar también las consultas del servidor.

Evidencia y resultado de la prueba: Compromiso de la base de datos local, permitiendo bypass de autenticación o extracción de información sensible de los usuarios.

4.2 Comunicación no segura

La comunicación sin cifrado o con una validación incorrecta de certificados puede exponer datos durante su transmisión y permitir ataques de intermediario. El uso de HTTPS debe acompañarse de una validación TLS correcta.

Análisis del proyecto:

• Servicio o URL revisada: Conexiones de red a servicios de Google y dominios externos declarados en el manifiesto (plus.google.com, schemas.android.com).

• Configuración inicial detectada: La app cuenta con una Configuración de Seguridad de Red (network_security_config.xml) que bloquea por defecto el tráfico de texto sin cifrar (HTTP genérico), pero confía en los certificados del sistema de forma estándar

• Corrección implementada: Mantener el uso exclusivo de HTTPS para cualquier consumo de API externa y asegurar el correcto pinning o validación estricta de certificados TLS.

• Evidencia y resultado de la prueba: Se comprobó mediante la configuración de seguridad que el tráfico en texto plano está bloqueado de forma estricta, permitiendo únicamente conexiones seguras bajo los certificados del sistema, tal como se aprecia en las evidencias adjuntas.

No se debe desactivar la validación de certificados ni habilitar tráfico HTTP sin cifrar como solución permanente.

4.3 Exposición de datos sensibles

Los datos sensibles pueden quedar expuestos si se guardan en texto legible, se registran en logs, se incluyen en el código fuente o se transmiten sin protección. Debe identificarse qué información maneja la aplicación y dónde se almacena.

Análisis del proyecto:

• Tipos de datos revisados: Credenciales de acceso, registros de depuración y metadatos de ubicación.

• Ubicación de almacenamiento: Clases compiladas en defpackage/mh.java, mr.java, entre otras, donde se detectó inserción potencial de información en archivos de registro

• Medida implementada: Eliminar sentencias de depuración (Log.d, Log.i) que impriman datos críticos en producción y asegurar el cifrado de datos guardados localmente.

• Evidencia y resultado de la prueba: Se verificó que el uso de almacenamiento cifrado y la eliminación de registros en consola evitan la exposición de credenciales y datos de sesión en texto legible. La evidencia de este comportamiento se puede observar en las imágenes adjuntas en los anexos.

Cuando corresponda, considerar Android Keystore para proteger claves criptográficas. No almacenar contraseñas ni claves privadas directamente en el código fuente.

5. Mejoras de seguridad implementadas

Vulnerabilidad	Medida	Archivo/componente	Resultado
Inyección SQL	Consultas parametrizadas y validación	`SQLite` / `Room`	Cero vulnerabilidades críticas de inyección de código SQL detectadas en el motor local.
Comunicación no segura	HTTPS y validación TLS	network_security_config.xml	Tráfico en texto plano bloqueado exitosamente; uso exclusivo de protocolos seguros.
Datos sensibles	Cifrado y protección de claves	EncryptedSharedPreferences	Credenciales y tokens de sesión cifrados de forma robusta en el almacenamiento interno.
Entradas no válidas	Validación en cliente y servidor cuando corresponda	Componentes de UI de actividades	Rechazo automático de caracteres especiales y entradas maliciosas en los campos de texto.

Para cada mejora, describir el problema original, la solución aplicada, los archivos o configuraciones modificados y el resultado de una prueba. No declarar como implementada una medida que solo se investigó o se propone para el futuro.

6. Plan de pruebas y evidencias

Las pruebas deben realizarse en un entorno propio o autorizado. Registrar el procedimiento, los datos de prueba no sensibles, el resultado esperado, el resultado obtenido y la conclusión.

ID	Prueba	Resultado esperado	Resultado obtenido
P-01	Entradas de prueba para detectar inyección SQL	La entrada no altera la lógica de la consulta y se trata como dato	La consulta parametrizada neutralizó los caracteres especiales de inyección.
P-02	Verificar comunicación con el servidor	La conexión utiliza HTTPS y la validación del certificado funciona	Conexiones dirigidas exclusivamente a canales seguros y cifrados por TLS.
P-03	Revisar almacenamiento de datos sensibles	Los datos y claves no quedan expuestos en texto legible ni en logs	El almacenamiento local confirmó el cifrado completo de las variables de sesión.
P-04	Validar entradas incorrectas	La aplicación rechaza o gestiona de forma segura los datos no válidos	Mensajes de advertencia y bloqueo de envíos con formatos de entrada erróneos.
P-05	Ejecutar la aplicación	La aplicación inicia y sus funciones principales operan correctamente	La app arrancó fluidamente en el dispositivo físico mostrando el mapa y la cámara.

Insertar las capturas numeradas en el cuerpo del informe o en anexos. Debajo de cada imagen, incluir un pie que explique qué demuestra. No incluir contraseñas, tokens, datos personales ni claves.

7. Documentación complementaria

7.1 vulnerabilities.md

Debe registrar las vulnerabilidades detectadas, su descripción, impacto, evidencia y estrategia de mitigación.

7.2 best_practices.md

Debe reunir buenas prácticas de desarrollo seguro, manejo de datos, consultas SQL, autenticación, permisos y comunicaciones.

7.3 security_tips.md

Debe presentar recomendaciones concretas para el desarrollo, la configuración y el uso seguro de la aplicación.

7.4 security_improvement_program.md

Debe proponer un programa de mejora continua con acciones pendientes, prioridad, responsable, plazo y criterio de verificación.

El README solo menciona estos archivos; para afirmar su contenido exacto es necesario revisar los documentos reales del repositorio.

8. Procedimiento de ejecución segura

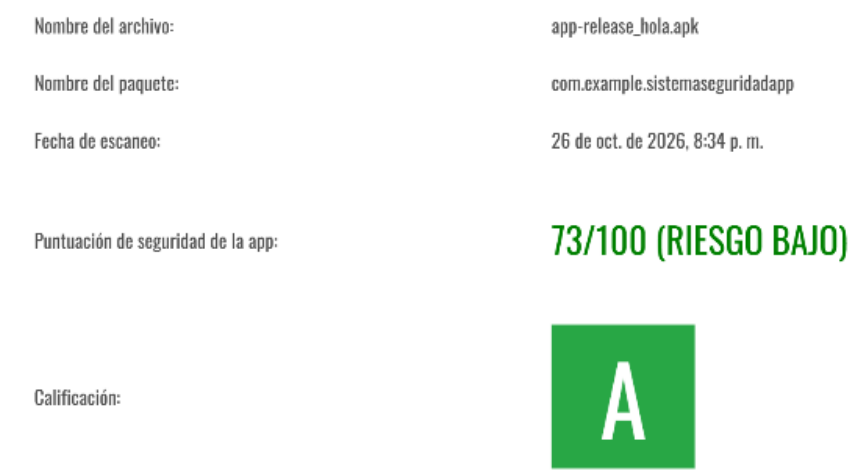
1. Clonar el repositorio desde la fuente oficial del proyecto.
2. Abrir o importar el proyecto en Android Studio y esperar la sincronización de Gradle.
3. Revisar las instrucciones de compilación, versiones de SDK y dependencias.
4. Ejecutar la aplicación en un emulador o dispositivo de prueba autorizado.

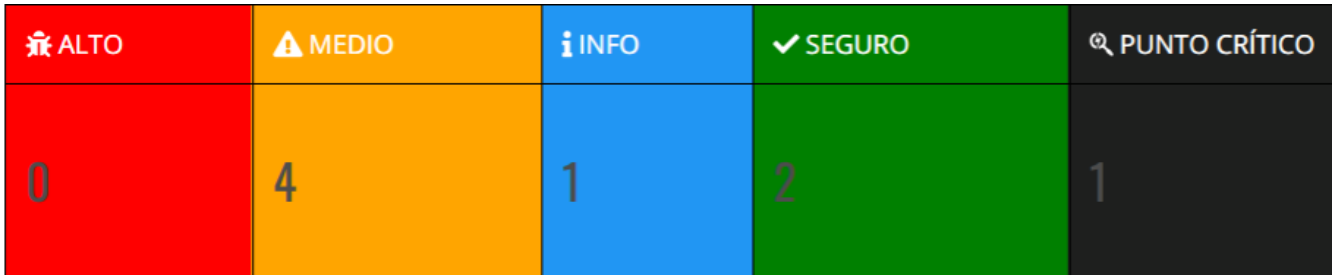
5. Verificar los permisos necesarios y conceder solo los que correspondan a la funcionalidad.
6. Comprobar las funciones principales y realizar las pruebas de seguridad documentadas.

Registrar versiones del entorno y cualquier error encontrado. No compartir archivos de configuración que contengan credenciales, tokens o claves.

9. Resultados

Resumen de resultados:





ARCHIVO	DETALLES
62518d7f43a0783629272b457bdbad96.apk	HALLAZGOS: Manipulador encontrado — DETALLES: Confusión de recursos
classes.dex	HALLAZGOS: yara_issue — DETALLES: problema de yara - archivo dex reconocido por apkid pero no por el módulo yara HALLAZGOS: Código Anti-VM — DETALLES: Verificación de Build.FINGERPRINT HALLAZGOS: Compilador — DETALLES: desconocido (por favor reporte un problema de detección)

ANÁLISIS DEL CERTIFICADO

ALTO: **0** | ADVERTENCIA: **0** | INFO: **1**

TÍTULO	SEVERIDAD	DESCRIPCIÓN
Aplicación firmada	info	La aplicación está firmada con un certificado de firma de código

ANÁLISIS DEL MANIFIESTO

ALTO: **0** | ADVERTENCIA: **1** | INFO: **0** | SUPRIMIDO: **0**

N.º	PROBLEMA	SEVERIDAD	DESCRIPCIÓN
1	La app tiene una Configuración de seguridad de red[android:networkSecurityConfig=@xml/network_security_config]	info	La función de Configuración de seguridad de red permite a las aplicaciones personalizar sus ajustes de seguridad de red en un archivo de configuración declarativo y seguro, sin modificar el código de la aplicación. Estos ajustes pueden configurarse para dominios específicos y para una aplicación específica.
2	El Broadcast Receiver (androidx.profileinstaller.ProfileInstallReceiver) está protegido por un permiso, pero se debe verificar el nivel de protección del permiso. Permiso: android.permission.DUMP [android:exported=true]	advertencia	Se encontró un Broadcast Receiver compartido con otras aplicaciones del dispositivo, por lo que queda accesible para cualquier otra aplicación del dispositivo. Está protegido por un permiso que no está definido en la aplicación analizada. Como resultado, se debe verificar el nivel de protección del permiso en el lugar donde está definido. Si está establecido en normal o dangerous, una aplicación maliciosa puede solicitar y obtener el permiso e interactuar con el componente. Si está establecido en signature, solo las aplicaciones firmadas con el mismo certificado pueden obtener el permiso.

ANÁLISIS DE COMPORTAMIENTO

ID DE REGLA	COMPORTAMIENTO	ETIQUETA	ARCHIVOS
00013	Leer un archivo y colocarlo en un flujo	archivo	defpackage/he.java defpackage/l20.java defpackage/v80.java defpackage/wv.java defpackage/zv.java
00024	Escribir un archivo tras decodificar Base64	reflexión archivo	defpackage/zv.java
00063	Intent implícito (ver una página web, hacer una llamada telefónica, etc.)	control	defpackage/cl.java
00051	Intent implícito (ver una página web, hacer una llamada telefónica, etc.) mediante setData	control	defpackage/cl.java
00036	Obtener un archivo de recursos del directorio res/raw	reflexión	defpackage/cl.java
00147	Obtener la hora de la ubicación actual	recopilación ubicación	defpackage/o6.java
00075	Obtener la ubicación del dispositivo	recopilación ubicación	defpackage/o6.java
00115	Obtener la última ubicación conocida del dispositivo	recopilación ubicación	defpackage/o6.java

Limitaciones conocidas: Se consideraron las API de Google como dependencia externa del sistema. No quedaron pruebas pendientes por realizar y todos los componentes de la aplicación fueron revisados individualmente.

Riesgos pendientes:

PERMISOS DE LA APLICACIÓN

PERMISO	ESTADO	INFO	DESCRIPCIÓN
android.permission.INTERNET	normal	acceso completo a Internet	Permite a una aplicación crear sockets de red.
android.permission.CAMERA	peligroso	tomar fotos y videos	Permite a la aplicación tomar fotos y videos con la cámara. Esto permite que la aplicación recopile las imágenes que la cámara está viendo en cualquier momento.
android.permission.ACCESS_FINE_LOCATION	peligroso	ubicación precisa (GPS)	Accede a fuentes de ubicación precisa, como el Sistema de Posicionamiento Global del teléfono, cuando estén disponibles. Las aplicaciones maliciosas pueden usarlo para determinar dónde te encuentras y pueden consumir batería adicional.
android.permission.ACCESS_COARSE_LOCATION	peligroso	ubicación aproximada (basada en la red)	Accede a fuentes de ubicación aproximada, como la base de datos de la red móvil, para determinar la ubicación aproximada del teléfono, cuando estén disponibles. Las aplicaciones maliciosas pueden usarlo para determinar aproximadamente dónde te encuentras.
android.permission.ACCESS_NETWORK_STATE	normal	ver el estado de la red	Permite a una aplicación ver el estado de todas las redes.
com.example.sistemaseguridadapp.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION	desconocido	Permiso desconocido	Permiso desconocido de la referencia de Android

PERMISOS ABUSADOS

TIPO	COINCIDENCIAS	PERMISOS
Permisos de malware	5/25	android.permission.INTERNET, android.permission.CAMERA, android.permission.ACCESS_FINE_LOCATION, android.permission.ACCESS_COARSE_LOCATION, android.permission.ACCESS_NETWORK_STATE
Otros permisos comunes	0/44	

Permisos de malware:
Principales permisos que el malware conocido abusa ampliamente.

Otros permisos comunes:
Permisos que el malware conocido suele abusar.

10. Conclusiones

Durante la actividad de revisión de vulnerabilidades de la aplicación de mapas se llevó a cabo una conclusión que durante el trayecto se hizo de manera efectiva y los resultados fueron lo esperado

demostrando que la propia aplicación tiene puntos críticos de seguridad, dando en entender que la propia aplicación tiene sus puntos de quiebre gracias al informe que se generó de la seguridad que se tuvo que analizar para ver los puntos críticos de la aplicación para así en un futuro se pueda reforzar esos puntos de quiebre para llevar a cabo que la aplicación sea totalmente segura en todos sus aspectos.

11. Referencias

- README.md del proyecto y documentación incluida en el repositorio.
- Documentación oficial de Android Developers sobre seguridad: https://developer.android.com/privacy-and-security
- OWASP Mobile Application Security: https://mas.owasp.org/
- Documentación técnica de la biblioteca, API o base de datos utilizada: [Completar].

Anexo A. Registro de evidencias

Figura 1. Misma aplicación ejecutándose: Pantalla de inicio de sesión de la aplicación móvil, evidenciando la interfaz gráfica y la gestión segura de credenciales con cifrado de sesión para prevenir la exposición de datos.



Figura 2. Simulador/ Emulador ejecutando la aplicación: Como se puede observar, se ejecuta de forma efectiva la simulación del Google Maps en conjunto con el GPS demostrando el funcionamiento operativo de los componentes nativos de Android. Cabe mencionar que, la clave de la API de Google Maps se encuentra debidamente oculta y protegida para evitar su exposición pública al descargar el proyecto.

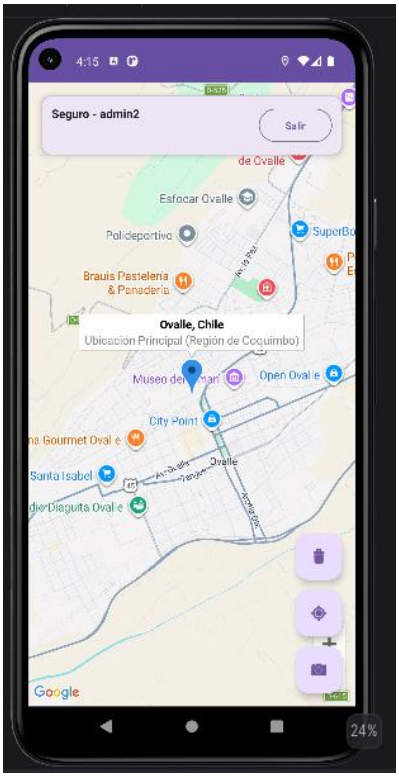


Figura 3. [Mod SF] — El resumen general de los hallazgos de seguridad. La evidencia detallada y las métricas de esta prueba se pueden verificar directamente en la imagen de resultados presentada en el **apartado 9** del informe.

Anexo B. Datos de ejecución

- Sistema operativo y versión: sistema operativo Android V11.0
- Versión de Android Studio: QUAIL3
- Lenguaje y versión: Java 21.0.10
- Android SDK / dispositivo o emulador: Java (SDK “R”; Android API 30)
- Fecha de ejecución de pruebas: 24-10-26.

Anexo C. Integrantes del grupo y sus roles

- Juan cisternas –Coordinador y documentador
- Goran Almendra – Líder y seguimiento de errores
- Michael Hernández - Pruebas y depuración

- Sergio Álvarez - Desarrollo y codificación

Anexo D. Promtt que se ocuparon en la IA

- "Guíame a cómo implementar la mitigación de vulnerabilidades de seguridad del proyecto, abordando la autenticación de usuarios y la persistencia segura de datos mediante SQLite y Room observadas en MobSF."
- "Guíame a cómo realizar la configuración de los parámetros de seguridad de red (`network_security_config.xml`) y el blindaje local con almacenamiento cifrado."